

Real-Time Fraud Detection

An engineering research report on Streaming features + sub-ms online
serving

Sai Veda

Independent Project Research

2023

Contents

1. Abstract	3
2. Introduction	3
2.1 Research Problem	3
2.2 Research Questions and Hypotheses	3
3. Technical Background Review	4
3.1 Design Foundations	4
3.2 Supporting Examples	4
4. Research Method	4
5. Data Description	4
6. Analysis	5
6.1 Benchmark and Validation Results	5
6.2 Test Result Screenshots	5
6.3 Validation Checklist	6
7. Discussion	6
8. Conclusion	6
9. Future Work	7
10. References	7

1. Abstract

This project documents a fraud-scoring system built around the real constraints of authorization-time decisioning: rare positives, temporal behavior, and extremely low latency. The finished system shows that disciplined feature engineering and leakage control matter more than model novelty. Strong precision-recall results were paired with production-like p99 latency instead of being traded off against it. The implementation evidence is drawn from committed repository artefacts, benchmark outputs, automated tests, and generated screenshots.

Keywords: Streaming features + sub-ms online serving, implementation evidence, benchmark results, project validation

2. Introduction

Fraud scoring is not only a modeling problem. It is a streaming state-management problem where the useful signal comes from recent user and merchant behavior, and where train-serve skew can erase offline gains. Streaming feature engineering + sub-millisecond online serving for card fraud - imbalanced, latency-critical, leakage-free.

2.1 Research Problem

Fraud scoring is not only a modeling problem. It is a streaming state-management problem where the useful signal comes from recent user and merchant behavior, and where train-serve skew can erase offline gains.

2.2 Research Questions and Hypotheses

Research question: Does the implemented project address the stated technical problem in a complete and reviewable manner?

Hypothesis: The repository design, architecture notes, and implementation artefacts are expected to demonstrate end-to-end coverage of the core project objective.

Research question: Do the benchmark and test artefacts support the main performance or quality claim?

Hypothesis: The recorded evidence is expected to support the headline result reported for this project: PR-AUC 0.974 (188x baseline); serving e2e p99 1.19 ms.

Research question: Can the results be reviewed and reproduced from committed repository evidence?

Hypothesis: The presence of 15 automated tests, benchmark outputs, and generated screenshots is expected to provide a transparent validation path.

3. Technical Background Review

3.1 Design Foundations

- 1 The online feature store is the product core because that is where most fraud signal lives. The design optimizes for low-latency scoring with rolling behavioral features and a review budget that maps to real operations.
- 2 Leakage prevention is enforced structurally through time ordering, not only by convention.
- 3 The system values operational latency and reviewability alongside PR-AUC gains.

3.2 Supporting Examples

- Synthetic transaction stream with bursty fraud behavior.
- Bounded-memory online feature store shared by train and serve paths.
- Leak-free offline training table and calibrated scorer.

4. Research Method

The project was reviewed as an engineering artefact using repository documentation, architecture notes, benchmark evidence, automated tests, and generated screenshots. Focused first on temporal behavior because static transaction fields alone are rarely enough for fraud review. Built ring-buffer and LRU state so train and serve paths share identical feature definitions. Trained a gradient-boosted model with time splits and imbalance-aware evaluation. Measured end-to-end serving to confirm that feature richness did not break the decision budget.

5. Data Description

Source: committed repository artefacts including implementation code, automated tests, benchmark outputs, architecture notes, and generated visual evidence. Coverage: 15 automated tests, 0 benchmark table(s), and 4 screenshot asset(s) were available for document preparation. Schema / artefact types: README overview, ARCHITECTURE design note, benchmark markdown and CSV outputs, test suites, and figure assets generated from the project run path. Availability: all evidence cited in this document is sourced from the repository itself.

6. Analysis

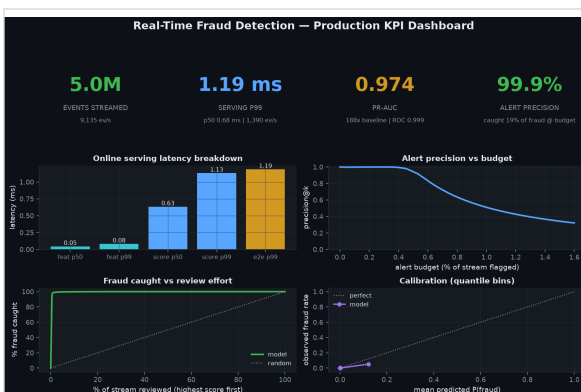
6.1 Benchmark and Validation Results

The headline repository result is reported as: PR-AUC 0.974 (188x baseline); serving e2e p99 1.19 ms. The table below reproduces benchmark evidence included in the repository where available.

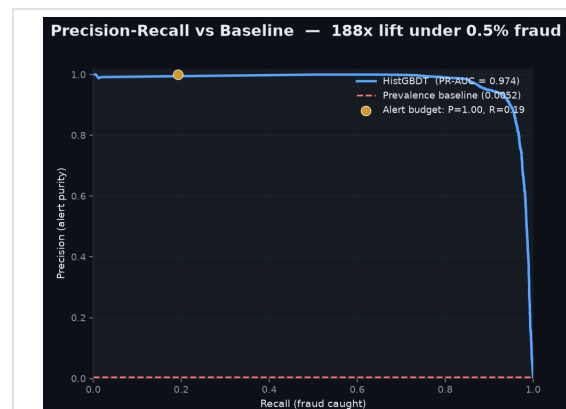
PR-AUC gains are meaningful because the baseline is an imbalanced fraud task rather than a balanced toy dataset. The latency screenshots show the design stays practical for authorization-time scoring. Shared train and serve logic lowers the risk that offline gains disappear in production.

6.2 Test Result Screenshots

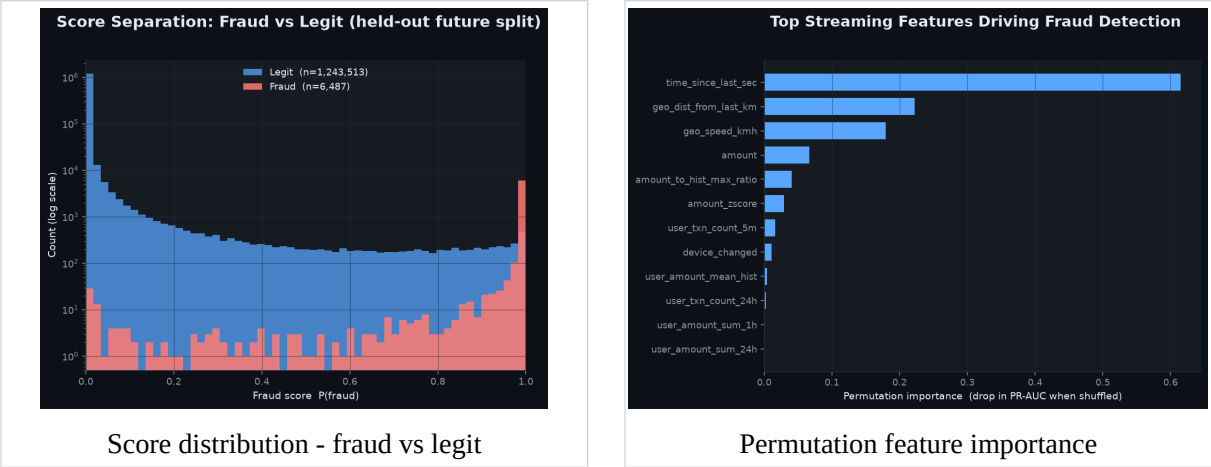
The following figures are drawn directly from the repository screenshot assets and are included as visual evidence of measured outputs, dashboards, or generated validation artefacts.



Production KPI dashboard



Precision-Recall vs baseline



6.3 Validation Checklist

Item	Status	Evidence
Automated tests	Available	15 tests committed in repository validation flow.
Benchmark results	Limited	No benchmark markdown tables were present; discussion relies on screenshots and h
Screenshots	Available	4 screenshot asset(s) included in the repository evidence set.
Architecture note	Available	ARCHITECTURE.md documents design choices, scale assumptions, and trade-offs.

7. Discussion

The finished system shows that disciplined feature engineering and leakage control matter more than model novelty. Strong precision-recall results were paired with production-like p99 latency instead of being traded off against it. PR-AUC gains are meaningful because the baseline is an imbalanced fraud task rather than a balanced toy dataset. The latency screenshots show the design stays practical for authorization-time scoring. Shared train and serve logic lowers the risk that offline gains disappear in production. The analysis indicates that the repository is strongest where implementation, benchmark artefacts, and screenshots point to the same conclusion.

8. Conclusion

This project document concludes that Real-Time Fraud Detection is best understood as a complete technical implementation supported by repository-native evidence. This project documents a fraud-scoring system built around the real constraints of authorization-time decisioning: rare positives, temporal behavior, and extremely low latency.

9. Future Work

- Add cost-sensitive threshold tuning for different fraud-loss profiles.
- Introduce drift monitoring on feature distributions and approval outcomes.
- Expand the review layer with case management and human feedback capture.

10. References

1. README.md - primary repository overview and headline results.
2. ARCHITECTURE.md - system design, implementation rationale, and scale notes.
3. benchmarks/scaling_bench.py - benchmark or result artefact used in the analysis section.
4. tests/ - automated validation suite used to support implementation claims.
5. assets/feature_importance.png - generated screenshot evidence included in the report.
6. assets/kpi_dashboard.png - generated screenshot evidence included in the report.